

Code Review — the 2026-07-27 audit changes

Scope: `aff6240...HEAD`, the six commits of the systems audit and thesis defence. 1,104 insertions, 94 deletions, 21 files. **Reviewer: the author.** Every claim below was run.

A self-review has an obvious conflict of interest, so this one is organised around finding defects in the audit's own output rather than restating what it fixed. Three were found. Two are fixed in this pass; one is recorded as a latent risk with its severity argued down honestly rather than up.

Summary

The changes fix four silent-wrongness defects and add four checks that fail on the pre-fix code. The diagnostic work is sound and every headline number was reproduced from a command. The *new* code is weaker than the fixes it delivers: it contained one instance of the exact defect class the audit was written to eliminate, one abuse of exceptions for control flow on a hot path, and one over-broad regex of the same kind the audit criticised in `selftest.js`.

Critical issues

#	File	Line	Issue	Severity
1	<code>engine/validate_selfplay.js</code>	356	INCONCLUSIVE incremented and never reported — a run with an unjudgeable check printed "16 passed, 0 failed" and read as clean	● Critical

Why critical. This is rule R5 from the architecture review — *never ran, passed and failed are three outcomes, not two* — violated in the same commit that introduced the rule, in the same file whose sample-floor fix exists to stop a check asserting on 8 games. The counter was incremented at line 190 and the summary at line 356 printed only `PASS` and `FAIL`. A counted-but-unshown outcome is exactly as invisible as an uncounted one.

Found by grepping my own diff for symbols that are written and never read — which is now the first thing I would do to any change of this shape.

Fixed. Verified:

SELF-PLAY VALIDATION: 16 passed, 0 failed, 1 inconclusive (too few observations to judge — not a pass)

Suggestions

#	File	Line	Suggestion	Category
2	engine/set_priors.js	525	Exception thrown to skip a block, on the common path	Correctness / Maintainability
3	engine/set_priors.js	269	observedSets() costs 721 ms and ~236 MB peak heap per process; 12 self-play workers pay it each	Performance
4	tests/run-all.js	52	looksLikeACheck matches prose in comments	Correctness (low)
5	tests/test-prng.js	70	new Function() on source lifted by brace-matching	Maintainability

2 — an exception used as a `goto`, and it hijacked a meaningful `catch`

```
try {
  if (drawn.length) throw new Error('already drawn from an observed set');
  const SM = require('./smogon_priors.js').forSpecies(species);
  ...
} catch (e) { /* fall through to the behaviour-clone */ }
```

Two problems, the second worse than the first. It allocates an `Error` — with a stack capture — on the *common* path, since most species now draw from an observed set. And it makes a deliberate skip indistinguishable from a genuine failure inside the Smogon lookup, because both land in a `catch` whose comment promises "fall through to the behaviour-clone". A real breakage in `smogon_priors` would have been silently absorbed as if it were a normal skip.

Fixed by skipping with a null instead: `const SM = drawn.length ? null : require(...)`. Behaviour verified identical — `test-set-realism.js` still reports the gap at +4.3 points, 6 passed / 0 failed.

3 — the observed-set store is loaded per process, and self-play runs twelve of them

Measured:

```
require: 3ms | first observedSets(): 721ms | heap after: 236MB | cached call: 0ms
```

The cost is `Q.loadGames()` on two stores (~50 MB of JSON) to extract ~2 MB of move arrays. It is lazy — a process that never generates a set pays nothing — and cached per process. But `mew_farm.js` shards across 12 workers, each of which generates sets, so a full farm pays **12 × 721 ms and up to ~2.8 GB of peak heap** for data that is identical in every worker.

Nothing observed has failed; the 9,646-game run completed in 6.1 minutes. So this is a suggestion, not a finding. **The right fix is a derived artifact:** have a build step write `data/observed-sets.json` (species → array of four-move arrays) and have `observedSets()` read that, falling back to the stores if it is missing or older than them. Workers then parse ~2 MB. That is also more S13-consistent than re-deriving the same thing twelve times. Not done tonight because a new build artifact introduces a staleness relationship that needs its own check, and adding an unverified check at 3am is how the `quality-filter.json` provenance block went stale.

4 — my unrun-check detector over-matches, which is the sin I criticised

```
const looksLikeACheck = src => /\d+ passed, \${F?}\}? ?failed|passed, .*failed/.test(src) || ...
```

Tested directly:

input	matches?
<code>` console.log( \${P} passed, \${F} failed ) `</code>	yes — correct
<code>/* the run reported 12 passed, 3 failed last night */</code>	yes — wrong

The second alternative `passed, .*failed` matches prose. This is the same defect I documented in `engine/selftest.js`, where grepping for a filename anywhere in a file counted three non-readers and inflated the GARBODOR debt from 12 to 17.

Severity is genuinely low, and I am not going to inflate it: the output is a *warning* about files nothing runs, not a pass/fail assertion, so a false positive costs a line of noise rather than a wrong verdict. It currently produces no false positives in this repository. It should still be tightened to require the pattern inside a template literal or a `console.log`, and comments stripped as `test-prng.js` does.

5 — `new Function()` on lifted source

`test-prng.js` extracts each generator from its file by brace-matching and evaluates it. This is deliberate — a re-typed copy of the generator would test the copy, not the shipped code, which is the same reasoning `test-mag-page.js` uses to lift the page's scoring functions. It is nonetheless arbitrary code execution from a file path, and the brace matcher is fragile: a generator written as a `function` declaration rather than `const name = ...` returns null. That case fails loudly ("could lift ... to test it"), which is the correct direction, so this is a maintainability note and not a bug.

Security

Nothing in this diff touches authentication, user input, network requests, deserialisation of untrusted data, or file paths derived from input. The one new argument-driven file read (`validate_selfplay.js` `argv[2]`) is `path.resolve`d and existence-checked, and this is a local research CLI with no untrusted caller. `new Function()` in `test-prng.js` executes repository source, not input. **No findings.**

Performance

Item 3 above is the only one. `assertDropped()` is $O(\text{rows} \times \text{candidates} \times |\text{DROP}|) \approx 82,836 \times \sim 9 \times 1$ over a one-off pass and adds under a second to a twelve-minute fit. `stab_audit.js` now draws one generated set per observed sheet rather than one per four, quadrupling its sampling work — deliberate, because the old 1:4 ratio put the confidence interval on the smaller sample.

Correctness — what I checked and found sound

- **The observed-set draw is frequency-weighted, correctly.** `cand[Math.floor(r() * cand.length)]` is uniform over the candidate *pool*, and repeated real sets appear repeatedly in that pool (1,949 Incineroar sets), so uniform-over-pool is weighted by empirical frequency. That is what should happen

and it happens by construction rather than by an explicit weighting step, so it is worth stating that it was verified rather than assumed.

- **Revealed moves are never displaced.** `observedDraw` returns only `out.slice(want.length)`, and `fillSet` concatenates onto the existing `moves`. A known move cannot be overwritten by a drawn set — which matters, because CLAUDE.md records "do not remove moves I chose" as a standing correction.
- **The `loadGames()` `TypeError` is a breaking change**, and the blast radius was measured before shipping: `grep -rn "loadGames(['\'])"` found exactly one caller passing a string, and the Python `load_games` equivalent had none.
- `assertDropped` **scans the whole corpus.** Asserted explicitly in the test by injecting an escaped row at index 417 of 600, because the precedent in this repo is a duplicate check that read the first 5,000 lines while all 401 duplicates sat past line 7,144.

Test coverage

Four new test files, 26 checks. Each was verified to fail on the pre-fix code, which is the property that distinguishes a test from a comment:

test	checks	verified against broken code
<code>test-drop-guard.js</code>	7	structural half counts <code>B.featuresFor()</code> call sites: 2 pre-fix, 1 post
<code>test-set-realism.js</code>	6	pre-fix sampler → 3 passed / 3 failed; fixed → 6 / 0
<code>test-prng.js</code>	7	pre-fix generators → 3 failures; fixed → 7 / 0
<code>test-mag-page.js</code> (extended)	+1	fails now, by design — the page implements 21 of 47 features

Suite: **19 passed, 3 failed, 1 skipped**. The three failures are pre-existing and documented (browser re-implementation, 12 GARBODOR readers, five stale living docs). None is masked.

What looks good

- Every fix carries the measurement that motivated it in a comment at the site, including the numbers. A future reader does not have to trust the changelog.
- The `RAW-STORE-NOT-READ` declaration keeps the GARBODOR guard strict while making its count honest, rather than loosening the guard to reduce the number — and the review records why the tempting fix (stripping strings before matching) would have created false negatives.
- `run-all.js` treating exit code 2 as "could not run" lets a gate whose corpus is gitignored stay listed instead of being quietly dropped from CI.

Verdict

Request changes — now addressed. Issue 1 was critical and is fixed; issue 2 is fixed; issues 3, 4 and 5 are recorded with severities argued rather than asserted, and 3 has a concrete design for its fix.

The uncomfortable result of this review is that the audit's own code reproduced the defect class it was written to eliminate — a value counted and never surfaced — within the same commit that codified the rule against it. That is not irony, it is evidence for the audit's central claim: this failure mode is not caused by carelessness, and writing the rule down is not sufficient to avoid it. Only a check is. The check that

caught it was reading my own diff for symbols written and never read, and that should become routine rather than incidental.